

Week 6: Product Review and Rating Management System

Scenario:

The e-commerce company wants to collect customer feedback about purchased products. Customers can provide ratings and reviews after completing their orders. The system should store customer opinions, analyze product performance, and identify highly rated products.

Students will create a **Product Review and Rating Module** connected with the existing Customer and Product modules.

Tasks to be Done:

1. Create Review and Rating Tables

```
SQL> CREATE TABLE Review(ReviewID VARCHAR2(7),
 2 CustomerID VARCHAR2(7),
 3 ProductID VARCHAR2(7),
 4 Rating NUMBER(1),
 5 ReviewText VARCHAR2(50),
 6 ReviewDate DATE,
 7 PRIMARY KEY(ReviewID),
 8 FOREIGN KEY(CustomerID) REFERENCES Customer(Customer_ID),
 9 FOREIGN KEY(ProductID) REFERENCES Product(Product_ID));
```

Table created.

```
SQL> desc Review
```

Name	Null?	Type
REVIEWID	NOT NULL	VARCHAR2(7)
CUSTOMERID		VARCHAR2(7)
PRODUCTID		VARCHAR2(7)
RATING		NUMBER(1)
REVIEWTEXT		VARCHAR2(50)
REVIEWDATE		DATE

Store Customer Feedback and Ratings

Tasks:

1. Add reviews for purchased products.

```
SQL> INSERT INTO REVIEW
  2 (REVIEWID, CUSTOMERID, PRODUCTID, RATING, REVIEWTEST, REVIEWDATE)
  3 VALUES
  4 ('REV0001', 'CUST000001', 'P000001', 5, 'Excellent laptop performance', DATE '2026-01-21');

1 row created.

SQL>
SQL> INSERT INTO REVIEW
  2 (REVIEWID, CUSTOMERID, PRODUCTID, RATING, REVIEWTEST, REVIEWDATE)
  3 VALUES
  4 ('REV0002', 'CUST000001', 'P000002', 4, 'Good wireless mouse', DATE '2026-01-21');

1 row created.

SQL>
SQL> INSERT INTO REVIEW
  2 (REVIEWID, CUSTOMERID, PRODUCTID, RATING, REVIEWTEST, REVIEWDATE)
  3 VALUES
  4 ('REV0003', 'CUST000002', 'P000004', 5, 'Very good sound quality', DATE '2026-02-02');

1 row created.

SQL>
SQL> INSERT INTO REVIEW
  2 (REVIEWID, CUSTOMERID, PRODUCTID, RATING, REVIEWTEST, REVIEWDATE)
  3 VALUES
  4 ('REV0004', 'CUST000002', 'P000006', 4, 'Good quality USB cable', DATE '2026-02-03');

1 row created.

SQL>
SQL> INSERT INTO REVIEW
  2 (REVIEWID, CUSTOMERID, PRODUCTID, RATING, REVIEWTEST, REVIEWDATE)
  3 VALUES
  4 ('REV0005', 'CUST000003', 'P000005', 5, 'Excellent smartphone', DATE '2026-02-15');

1 row created.
```

SQL> SELECT * FROM Review;				
REVIEWI	CUSTOMERID	PRODUCT	RATING	
REVIEWTEST				REVIEWDAT
REV0001	CUST000001	P000001	5	
Excellent laptop performance				20-JAN-26
REV0002	CUST000001	P000002	4	
Good wireless mouse				21-JAN-26
REV0003	CUST000002	P000004	5	
Very good sound quality				02-FEB-26
REVIEWI	CUSTOMERID	PRODUCT	RATING	
REVIEWTEST				REVIEWDAT
REV0004	CUST000002	P000006	4	
Good quality USB cable				03-FEB-26
REV0005	CUST000003	P000005	5	
Excellent smartphone				15-FEB-26
REV0006	CUST000004	P000008	4	
Useful smart watch				28-FEB-26
REVIEWI	CUSTOMERID	PRODUCT	RATING	
REVIEWTEST				REVIEWDAT
REV0007	CUST000004	P000006	5	
Cable works perfectly				01-MAR-26
REV0008	CUST000005	P000010	4	
Good tablet for daily use				12-MAR-26
REV0009	CUST000006	P000009	5	
Clear and comfortable headphones				25-MAR-26
REVIEWI	CUSTOMERID	PRODUCT	RATING	
REVIEWTEST				REVIEWDAT
REV0010	CUST000006	P000015	4	
Good sound and battery life				26-MAR-26
REV0011	CUST000007	P000008	5	
Excellent features				10-APR-26
REV0012	CUST000008	P000012	4	
Reliable storage device				30-APR-26
REVIEWI	CUSTOMERID	PRODUCT	RATING	
REVIEWTEST				REVIEWDAT
REV0013	CUST000008	P000006	3	
Average quality				01-MAY-26
REV0014	CUST000010	P000012	5	
Very useful external drive				28-MAY-26
REV0015	CUST000011	P000009	4	
Good headphones				15-JUN-26
REVIEWI	CUSTOMERID	PRODUCT	RATING	
REVIEWTEST				REVIEWDAT
REV0016	CUST000011	P000015	5	
Excellent earbuds				16-JUN-26
REV0017	CUST000012	P000014	4	
Useful laptop stand				01-JUL-26
REV0018	CUST000012	P000003	5	
Excellent keyboard				02-JUL-26
18 rows selected.				

2. Store customer ratings from 1 to 5 stars.

Ensure this by adding Constraint

```
SQL> ALTER TABLE Review ADD CONSTRAINT CHK_Rate CHECK(Rating >= 1 AND Rating <= 5);
Table altered.
```

```
SQL> INSERT INTO REVIEW
  2 VALUES ('REV0019', 'CUST000001', 'P000001', 6,
  3          'Excellent product', SYSDATE);
INSERT INTO REVIEW
*
ERROR at line 1:
ORA-02290: check constraint (C##ECOMMERCE.CHK_RATE) violated
```

3. Update review comments if required.

```
SQL> UPDATE REVIEW
  2 SET REVIEWTEST = 'Good quality but could be improved'
  3 WHERE REVIEWID = 'REV0013';

1 row updated.
```

4. Remove inappropriate or invalid reviews.

```
SQL> DELETE FROM REVIEW
  2 WHERE REVIEWID = 'REV0018';

1 row deleted.
```

4. Retrieve Product Review Details

Write SQL queries to:

1. Display all reviews for a product.

```
SQL> SELECT * FROM Review WHERE ProductID = 'P000012';
```

REVIEWID	CUSTOMERID	PRODUCT	RATING	REVIEWTEST	REVIEWDAT
REV0012	CUST000008	P000012	4	Reliable storage device	30-APR-26
REV0014	CUST000010	P000012	5	Very useful external drive	28-MAY-26

2. Display customer name with their reviews.

```
SQL> SELECT CONCAT(c.First_Name,c.Last_Name) AS Customer_Name,
2 R.ReviewID,R.ProductID,R.Rating,R.ReviewTest
3 FROM Customer c JOIN Review R
4 ON c.Customer_ID = R.CustomerID;
```

CUSTOMER_NAME	REVIEWI	PRODUCT	RATING
REVIEWTEST			

ArjunKumar Excellent laptop performance	REV0001	P000001	5
--	---------	---------	---

ArjunKumar Good wireless mouse	REV0002	P000002	4
-----------------------------------	---------	---------	---

PriyaSharma Very good sound quality	REV0003	P000004	5
--	---------	---------	---

CUSTOMER_NAME	REVIEWI	PRODUCT	RATING
REVIEWTEST			

PriyaSharma Good quality USB cable	REV0004	P000006	4
---------------------------------------	---------	---------	---

RahulMenon Excellent smartphone	REV0005	P000005	5
------------------------------------	---------	---------	---

AnanyaIyer Useful smart watch	REV0006	P000008	4
----------------------------------	---------	---------	---

CUSTOMER_NAME	REVIEWI	PRODUCT	RATING
REVIEWTEST			

PriyaSharma Good quality USB cable	REV0004	P000006	4
---------------------------------------	---------	---------	---

RahulMenon Excellent smartphone	REV0005	P000005	5
------------------------------------	---------	---------	---

AnanyaIyer Useful smart watch	REV0006	P000008	4
----------------------------------	---------	---------	---

CUSTOMER_NAME	REVIEWI	PRODUCT	RATING
REVIEWTEST			

AnanyaIyer Cable works perfectly	REV0007	P000006	5
-------------------------------------	---------	---------	---

VikramReddy Good tablet for daily use	REV0008	P000010	4
--	---------	---------	---

MeeraKrishnan Clear and comfortable headphones	REV0009	P000009	5
---	---------	---------	---

CUSTOMER_NAME	REVIEWI	PRODUCT	RATING
REVIEWTEST			

MeeraKrishnan Good sound and battery life	REV0010	P000015	4
--	---------	---------	---

SureshNair Excellent features	REV0011	P000008	5
----------------------------------	---------	---------	---

KavyaRajan Reliable storage device	REV0012	P000012	4
---------------------------------------	---------	---------	---

CUSTOMER_NAME	REVIEWI	PRODUCT	RATING
REVIEWTEST			
MeeraKrishnan	REV0010	P000015	4
Good sound and battery life			
SureshNair	REV0011	P000008	5
Excellent features			
KavyaRajan	REV0012	P000012	4
Reliable storage device			
CUSTOMER_NAME	REVIEWI	PRODUCT	RATING
REVIEWTEST			
KavyaRajan	REV0013	P000006	3
Good quality but could be improved			
DivyaPillai	REV0014	P000012	5
Very useful external drive			
AdityaVerma	REV0015	P000009	4
Good headphones			
CUSTOMER_NAME	REVIEWI	PRODUCT	RATING
REVIEWTEST			
AdityaVerma	REV0016	P000015	5
Excellent earbuds			
SnehaPatel	REV0017	P000014	4
Useful laptop stand			

17 rows selected.

- Find products having maximum reviews.

```
SQL> SELECT
  2     P.PRODUCT_ID,
  3     P.PRODUCT_NAME,
  4     COUNT(R.REVIEWID) AS REVIEW_COUNT
  5 FROM PRODUCT P
  6 JOIN REVIEW R
  7     ON P.PRODUCT_ID = R.PRODUCTID
  8 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  9 ORDER BY REVIEW_COUNT DESC;
```

PRODUCT	PRODUCT_NAME	REVIEW_COUNT
P000006	USB Cable	3
P000008	Smart Watch	2
P000012	External Hard Disk	2
P000015	Wireless Earbuds	2
P000009	Headphones	2
P000014	Laptop Stand	1
P000010	Tablet	1
P000005	Smartphone	1
P000001	Laptop	1
P000004	Bluetooth Speaker	1
P000002	Wireless Mouse	1

11 rows selected.

- Display recent customer feedback.

```

SQL> SELECT
  2     C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  3     P.PRODUCT_NAME,
  4     R.RATING,
  5     R.REVIEWTEST AS CUSTOMER_FEEDBACK,
  6     R.REVIEWDATE AS REVIEW_DATE
  7 FROM REVIEW R
  8 JOIN CUSTOMER C
  9     ON R.CUSTOMERID = C.CUSTOMER_ID
 10 JOIN PRODUCT P
 11     ON R.PRODUCTID = P.PRODUCT_ID
 12 ORDER BY R.REVIEWDATE DESC
 13 FETCH FIRST 5 ROWS ONLY;

```

CUSTOMER_NAME	PRODUCT_NAME	RATING	CUSTOMER_FEEDBACK	REVIEW_DA
Sneha Patel	Laptop Stand	4	Useful laptop stand	01-JUL-26
Aditya Verma	Wireless Earbuds	5	Excellent earbuds	16-JUN-26

CUSTOMER_NAME	PRODUCT_NAME	RATING	CUSTOMER_FEEDBACK	REVIEW_DA
Aditya Verma	Headphones	4	Good headphones	15-JUN-26
Divya Pillai	External Hard Disk	5		

CUSTOMER_NAME	PRODUCT_NAME	RATING	CUSTOMER_FEEDBACK	REVIEW_DA
	Very useful external drive			28-MAY-26
Kavya Rajan	USB Cable	3	Good quality but could be improved	01-MAY-26

5. Retrieve reviews with ratings above 4.

```
SQL> SELECT * FROM Review WHERE Rating > 4;
```

REVIEWID	CUSTOMERID	PRODUCT	RATING	REVIEWDATE
REV0001	CUST000001	P000001	5	20-JAN-26
REV0003	CUST000002	P000004	5	02-FEB-26
REV0005	CUST000003	P000005	5	15-FEB-26
REV0007	CUST000004	P000006	5	01-MAR-26
REV0009	CUST000006	P000009	5	25-MAR-26
REV0011	CUST000007	P000008	5	10-APR-26
REV0014	CUST000010	P000012	5	28-MAY-26
REV0016	CUST000011	P000015	5	16-JUN-26

8 rows selected.

Example Query:

5. Calculate Average Product Ratings

Use aggregate functions to analyze ratings.

Tasks:

1. Calculate average rating for each product.

```
SQL> SELECT p.Product_Name, AVG(R.Rating) AS Average_Rating
2 FROM Product p JOIN Review R
3 ON p.Product_ID = R.ProductID
4 GROUP BY p.Product_Name;
```

PRODUCT_NAME	AVERAGE_RATING
Laptop	5
Wireless Mouse	4
Bluetooth Speaker	5
Smartphone	5
USB Cable	4
Smart Watch	4.5
Headphones	4.5
Tablet	4
External Hard Disk	4.5
Laptop Stand	4
Wireless Earbuds	4.5

11 rows selected.

2. Count the number of reviews per product.

```
SQL> SELECT p.Product_Name, Count(R.Rating) AS Count_Rating
2 FROM Product p JOIN Review R
3 ON p.Product_ID = R.ProductID
4 GROUP BY p.Product_Name;

PRODUCT_NAME                                COUNT_RATING
-----
Laptop                                       1
Wireless Mouse                             1
Bluetooth Speaker                           1
Smartphone                                  1
USB Cable                                   3
Smart Watch                                 2
Headphones                                  2
Tablet                                      1
External Hard Disk                           2
Laptop Stand                               1
Wireless Earbuds                             2

11 rows selected.
```

3. Find highest-rated products.

```
SQL> SELECT
2     P.PRODUCT_ID,
3     P.PRODUCT_NAME,
4     ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING
5 FROM PRODUCT P
6 JOIN REVIEW R
7     ON P.PRODUCT_ID = R.PRODUCTID
8 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
9 HAVING AVG(R.RATING) = (
10     SELECT MAX(AVG_RATING)
11     FROM (
12         SELECT AVG(RATING) AS AVG_RATING
13         FROM REVIEW
14         GROUP BY PRODUCTID
15     )
16 );

PRODUCT PRODUCT_NAME                                AVERAGE_RATING
-----
P000001 Laptop                                       5
P000004 Bluetooth Speaker                           5
P000005 Smartphone                                  5
```

4. Identify products with average rating above 4.

```
SQL> SELECT
2     P.PRODUCT_ID,
3     P.PRODUCT_NAME,
4     ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING
5 FROM PRODUCT P
6 JOIN REVIEW R
7     ON P.PRODUCT_ID = R.PRODUCTID
8 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
9 HAVING AVG(R.RATING) > 4
10 ORDER BY AVERAGE_RATING DESC;

PRODUCT PRODUCT_NAME                                AVERAGE_RATING
-----
P000005 Smartphone                                  5
P000004 Bluetooth Speaker                           5
P000001 Laptop                                       5
P000012 External Hard Disk                           4.5
P000009 Headphones                                  4.5
P000015 Wireless Earbuds                             4.5
P000008 Smart Watch                                 4.5

7 rows selected.
```

6. Identify Highly Rated Products

Generate product rating reports.

Report 1: Product Rating Analysis

Display:

- Product name
- Number of reviews
- Average rating

```
SQL> SELECT
2     P.PRODUCT_NAME,
3     COUNT(R.REVIEWID) AS NUMBER_OF_REVIEWS,
4     ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING
5 FROM PRODUCT P
6 LEFT JOIN REVIEW R
7     ON P.PRODUCT_ID = R.PRODUCTID
8 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
9 ORDER BY AVERAGE_RATING DESC NULLS LAST;
```

PRODUCT_NAME	NUMBER_OF_REVIEWS
AVERAGE_RATING	
Bluetooth Speaker	1
5	
Smartphone	1
5	
Laptop	1
5	
PRODUCT_NAME	NUMBER_OF_REVIEWS
AVERAGE_RATING	
Smart Watch	2
4.5	
Wireless Earbuds	2
4.5	
External Hard Disk	2
4.5	
PRODUCT_NAME	NUMBER_OF_REVIEWS
AVERAGE_RATING	
Headphones	2
4.5	
USB Cable	3
4	
Wireless Mouse	1
4	
PRODUCT_NAME	NUMBER_OF_REVIEWS
AVERAGE_RATING	
Laptop Stand	1
4	
Tablet	1
4	
Mechanical Keyboard	0
PRODUCT_NAME	NUMBER_OF_REVIEWS
AVERAGE_RATING	
Web Camera	0
Gaming Monitor	0
Power Bank	0

15 rows selected.

Report 2: Customer Feedback Analysis

Find:

- Most reviewed products

```
SQL> SELECT
2     P.PRODUCT_NAME,
3     COUNT(R.REVIEWID) AS REVIEW_COUNT
4 FROM PRODUCT P
5 JOIN REVIEW R
6     ON P.PRODUCT_ID = R.PRODUCTID
7 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
8 ORDER BY REVIEW_COUNT DESC;
```

PRODUCT_NAME	REVIEW_COUNT
USB Cable	3
Smart Watch	2
External Hard Disk	2
Wireless Earbuds	2
Headphones	2
Laptop Stand	1
Tablet	1
Smartphone	1
Laptop	1
Bluetooth Speaker	1
Wireless Mouse	1

11 rows selected.

- Highly rated products

```
SQL> SELECT
  2   P.PRODUCT_NAME,
  3   ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING
  4 FROM PRODUCT P
  5 JOIN REVIEW R
  6   ON P.PRODUCT_ID = R.PRODUCTID
  7 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8 HAVING AVG(R.RATING) > 4
  9 ORDER BY AVERAGE_RATING DESC;
```

PRODUCT_NAME	AVERAGE_RATING
Smartphone	5
Bluetooth Speaker	5
Laptop	5
External Hard Disk	4.5
Headphones	4.5
Wireless Earbuds	4.5
Smart Watch	4.5

7 rows selected.

● Products requiring improvement

```
SQL> SELECT
  2   P.PRODUCT_NAME,
  3   COUNT(R.REVIEWID) AS REVIEW_COUNT,
  4   ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING
  5 FROM PRODUCT P
  6 JOIN REVIEW R
  7   ON P.PRODUCT_ID = R.PRODUCTID
  8 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  9 HAVING AVG(R.RATING) < 3
 10 ORDER BY AVERAGE_RATING;
```

no rows selected

Report 3: Rating Distribution

● Number of 5-star ratings

```
SQL> SELECT COUNT(*) AS FIVE_STAR_RATINGS
  2 FROM REVIEW
  3 WHERE RATING = 5;
```

FIVE_STAR_RATINGS
8

● Number of 4-star ratings

```
SQL> SELECT COUNT(*) AS FOUR_STAR_RATINGS
  2 FROM REVIEW
  3 WHERE RATING = 4;
```

FOUR_STAR_RATINGS
8

● Number of low-rated products

```
SQL> SELECT COUNT(*) AS LOW_RATED_PRODUCTS
  2 FROM (
  3   SELECT PRODUCTID
  4   FROM REVIEW
  5   GROUP BY PRODUCTID
  6   HAVING AVG(RATING) < 3
  7 );
```

LOW_RATED_PRODUCTS
0

● Complete Rating

```
SQL> SELECT
  2   RATING,
  3   COUNT(*) AS NUMBER_OF_RATINGS
  4 FROM REVIEW
  5 GROUP BY RATING
  6 ORDER BY RATING DESC;
```

RATING	NUMBER_OF_RATINGS
5	8
4	8
3	1